
AIRBRUSH

DRAWING GAME WITH WEBRTC AND MOTION-
SENSOR CONTROLS

ABAN RADWAN JERIDEH , 201805377

EDUARD MUNTEANU , 202108177

ENOK MALIK MAJ , 202005732

PÉTER GERGELY , 202402636

OSCAR

PROJECT REPORT

December 2024

Advisor: Niels Olof Bouvin



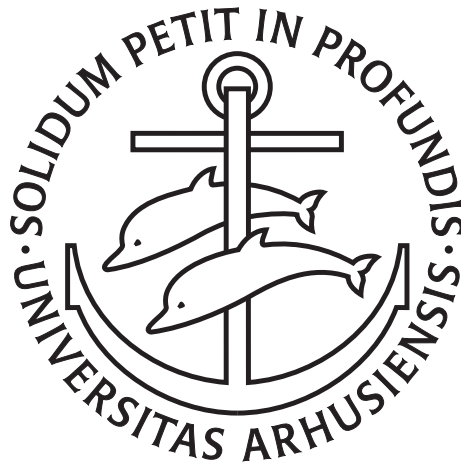
AARHUS
UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

AIRBRUSH

Drawing Game with WebRTC and Motion-Sensor Controls

ABAN JERIDEH, EDUARD MUNTEANU, ENOK MAJ & PÉTER GERGELY



Project Report

Department of Computer Science
Faculty of Natural Sciences
Aarhus University

December 2024

CONTENTS

1	Abstract	1
2	Introduction	3
3	Related Work	5
4	Methods	7
4.1	Analysis	7
4.1.1	Game Explanation	7
4.1.2	Software Architecture	8
4.1.3	Sensor Overview	9
4.1.4	General Architecture	10
4.1.5	Implementation	14
4.1.6	Evaluation	30
5	Results	35
6	Discussion	37
6.0.1	Conclusion	37
	Bibliography	39

LIST OF FIGURES

Figure 4.1	UML Diagram of Airbrush Architecture.	8
Figure 4.2	peers connected to the game	30
Figure 4.3	SignalingServer Logs connecting peers	31
Figure 4.4	logs of data channels with other peers	31
Figure 4.5	Demonstration of chat function peer 1.	32
Figure 4.6	Demonstration of chat function peer 2.	32
Figure 4.7	message data exchanged	32
Figure 4.8	Drawing function with motion data; drawing peer.	33
Figure 4.9	Canvas view from guessing peer.	33
Figure 4.10	Canvas view from 2nd guessing peer.	33

LIST OF TABLES

LISTINGS

4.1	signalingServer setup	16
4.2	signalingServer class	16
4.3	signalingServer ICE server	18
4.4	communication ICE server code	19
4.5	Communication ICE server	19
4.6	Communication setupDataChannel	20
4.7	Communication setupDataChannel	21
4.8	setupDataChannel	22
4.9	getChatBox from Guess	23

4.10 addChatBox from Guess	24
--------------------------------------	----

ACRONYMS

1

ABSTRACT

This project goes over the details of the IoT project of AirBrush by the Oscar group. We implemented a P2P system where you are able to draw and guess other people's drawings. This strives to be a holistic game product utilizing a series of technologies like sensors and cloud website hosting. Some measurements we wanted to keep down is delay and keep usability high for a entertaining user experience.

2

INTRODUCTION

Airbrush started and ended as a very ambitious project. Too ambitious. But the idea was something so captivating that we hardly could have picked a different project. Thus, it came about that we started developing the components needed to "draw in the air," such as capturing accelerometer input and figuring out which communication protocol would fit such a feat. Regardless of its completion, the project brings together a series of fascinating technologies indicative of the field of Internet of Things (IoT).

In essence, the vision of Airbrush was that of letting the user use their phone as a paintbrush in their actual surroundings together with friends in some game(s). The strokes of the brush, determined by the motion of the phone, appear on a shared virtual canvas accessible via a browser. Users can collaborate or compete in a variety of games using this augmented reality experience. However, several challenges presented themselves immediately: managing sensor accuracy, ensuring low-latency communication, and synchronizing actions between multiple peers. These obstacles, while daunting, also highlighted the potential learning value of the project. With guidance from our professor and contributions from every team member, we designed and implemented a cloud-hosted platform that leverages WebRTC for real-time peer-to-peer communication. This system connects users and enables them to draw, guess, and interact using their phones' motion sensors. While we encountered limitations along the way, such as achieving a perfect balance between performance and responsiveness, the project demonstrate how simple technologies can create a bigger whole. The project combines several contemporary technologies in a way that highlights their potential. WebRTC, a real-time peer-to-peer communication protocol, ensures that players can connect and interact without relying on centralized servers. By processing motion data from accelerometers and gyroscopes, smartphones are transformed into interactive tools for drawing. Despite challenges such as sensor jitter and NAT traversal issues, the system provides a robust and scalable foundation for collaborative gameplay.

Not all is doom and gloom, however. The idea is too interesting to us to let go, and so the website is intended for being kept around and if time and JavaScript permits, may be continued as a side-project.

In this report we document our efforts in making this project a reality with worked and what didn't. we begin by laying out the related work that inspired the project, followed by an exploration of the architecture

and implementation choices, and concludes with an evaluation of the system's strengths, limitations, and potential. Airbrush is not just a system but an exploration of how IoT and decentralized communication can enable new forms of interaction and play.

Gitlab link: <https://gitlab.au.dk/iot-oscar/airbrush>

Youtube link (may be used for anything): https://www.youtube.com/shorts/Ku_RMxFQ12E

3

RELATED WORK

During the development of Airbrush we made explorations of various technologies and systems that address challenges in decentralization, real-time interaction, and motion-based input. Among the most direct influences was Scribbl.io[5], a collaborative drawing game where players take turns sketching on a shared canvas while others guess the word being illustrated. Scribbl.io demonstrates the appeal of combining simple mechanics with collaborative gameplay, making drawing a social and interactive experience. However, it relies on a centralized server architecture and we wanted to apply the concepts of decentralised p2p development when it comes to Airbrush.

Scribbl.io operates on a traditional server-client model. A central server handles all connections, synchronizes game states, and manages user interactions. While this approach ensures reliability and simplifies implementation, it also introduces vulnerabilities, including single points of failure and inherent latency caused by routing all communication through the server. These limitations motivated us to use a peer-to-peer architecture in Airbrush, where direct connections between players could enhance scalability and reduce communication delays. By removing the dependency on centralized infrastructure, Airbrush aimed to create a more resilient and efficient system for collaborative interaction.

The foundational principles of decentralized systems were drawn from peer-to-peer communication research. One of the most comprehensive analyses of P2P technologies is provided by Androutsellis-Theotokis and Spinellis in their survey of peer-to-peer content distribution systems[1]. Their work categorizes P2P architectures into unstructured, structured, and hybrid systems, each with its strengths and trade-offs. For Airbrush, the ability of structured P2P architectures to ensure predictable data routing and efficient communication was particularly relevant. These insights informed how Airbrush manages synchronization and data exchange between peers, enabling consistent canvas updates without a central server. The survey also highlights challenges such as security and maintaining state consistency across nodes, issues that were carefully considered during Airbrush's development.

WebRTC served as the backbone for implementing Airbrush's peer-to-peer communication. As described by Loreto and Romano[4], WebRTC provides a robust framework for establishing real-time connections between devices. Its ability to handle NAT traversal using STUN and TURN servers, combined with its encrypted data channels, made it the ideal choice for transmitting motion data and canvas updates. The

flexibility of WebRTC to support both high-bandwidth media streams and lightweight data packets allowed Airbrush to maintain smooth gameplay while handling the demands of real-time interaction. This technology bridged the gap between the conceptual and technical goals of Airbrush, ensuring that the system could function without compromising responsiveness or reliability.

The motion-based interaction central to Airbrush leverages smartphones' accelerometers and gyroscopes to translate physical movements into virtual brushstrokes. This approach draws upon the foundational work of Hinckley et al.[2] in "Sensing Techniques for Mobile Interaction," which explores integrating sensors into handheld devices to enhance interaction. Their research highlights the potential of sensors to detect device orientation, movement, and proximity, enabling more intuitive user experiences. Applying these principles, Airbrush utilizes motion sensing to capture users' gestures, allowing for dynamic and engaging drawing experiences.

However, implementing this aspect of Airbrush proved challenging due to the inherent inaccuracy of smartphone sensors. Noise and drift in accelerometer and gyroscope data often resulted in irregularities in the virtual brushstrokes, undermining the fluidity of the drawing experience. To address this, filtering algorithms were employed to process and refine the sensor data, smoothing out inconsistencies and ensuring that virtual brushstrokes more closely reflected users' intended movements. While this approach mitigated some of the issues, achieving a fully seamless and responsive interface remains a technical hurdle for motion-based systems relying on consumer-grade hardware.

While Airbrush currently relies on WebRTC for real-time communication, MQTT presents an interesting alternative and a model we learned a lot from, for managing IoT-based data transmission. As outlined by Hunkeler et al.[3], MQTT's publish/subscribe model offers a lightweight and efficient approach to handling intermittent connections and constrained network environments. Although not directly integrated into Airbrush, MQTT's principles highlight opportunities for optimizing data exchange in future iterations, particularly in scenarios where network reliability varies.

The works explored here demonstrate the interconnected challenges of decentralization, real-time communication, and user interaction that define the Airbrush project. Scribbl.io provided the conceptual groundwork, while the theoretical and technical foundations of P2P systems, WebRTC, and gesture-based interfaces shaped its architecture and implementation. These influences collectively informed the decisions that brought Airbrush to its current state.

4

METHODS

"Who in the world am I? Ah, *that's* the great puzzle!"

Alice's Adventures in Wonderland
LEWIS CARROLL

4.1 ANALYSIS

The Airbrush project combines creativity, competition, and technology in a unique multiplayer browser-based game. This section analyzes the key components of the system, including the game mechanics, software architecture, communication flow, and challenges encountered during development.

4.1.1 Game Explanation

Airbrush is a captivating multiplayer browser-based video game that challenges players to showcase their creativity and quick thinking. In this innovative game, one player takes on the role of the drawer, tasked with illustrating a secret word while other players attempt to guess what is being depicted. The twist lies in the unique drawing mechanic: the drawer must use their device to create air gestures, mimicking the movements of an airbrush or graffiti artist to render their artwork on the digital canvas.

At the start of each round, the drawer selects a secret word from four given options. As the round progresses and the timer counts down, hints about the secret word are gradually revealed to all players. The faster a player guesses the correct word, the more points they earn, rewarding both speed and intuition. When a player successfully identifies the word, their score is updated based on how quickly they arrived at the answer.

Once the round concludes, the drawing responsibilities are passed to the next player, ensuring that everyone gets a turn to draw and guess. This dynamic gameplay loop continues, with points accumulating across rounds to determine the ultimate winner. Airbrush seamlessly combines creativity, competition, and fun, making it an engaging experience for players of all skill levels.

4.1.2 Software Architecture

Regarding our product structure, we will provide with a UML diagram which shows the relation between the classes.

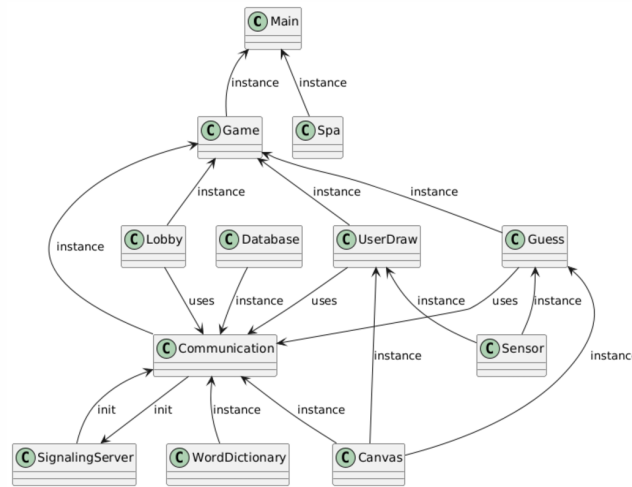


Figure 4.1: UML Diagram of Airbrush Architecture.

The simplified UML diagram shows the general relationships, omitting less relevant relations in order to not obscure the users visualisation, illustrates our prototype overall software architecture. The Main class is responsible to initialize the Game using the SPA (Single-Page Application). More concrete, which is omitted in the UML for simplification reasons, the SPA creates a Game instance which is responsible to initialize the Lobby. The user accessing the website will observe the lobby session, where he simply needs to input his username, pressing on the play-button to get in the game. The Game has the responsibility of dictating users roles while interacting with the website. More specifically, the Game, assisted by the SPA, decide if the user should be in the Lobby, UserDraw or Guess section, depending on the peers interaction with the application but also the interaction between themselves. SPA purpose is to change the page HTML script based on the Game class decisions. We decided to make our website as a Single-Page Application due to it's simplicity and flexibility, but especially because of the data and communication reset issues, which will be further discussed in the implementation section. UserDraw and Guess responsibilities have the same pattern. They are responsible on different aspects of the game, using Communication to send the updates back and forth with the other peers, providing a consistent visualization for all peers connected to the network. Lobby is pretty straightforward. It's only responsibility is to help the peer connecting to the network. The username purpose it's to provide a more human friendly identification. Communication is the central core. If the communication didn't exist, the peers wouldn't be able to interact

with each other, having the P2P network being unexisting. Each peer from the P2P network is provided with a Database in which the data received by the other peers or by his own interactions are stored. The peer database is an instance in the Communication class for updating the database given data received from the other peers, as well as being an instance in UserDraw and Guess, updating the database based on the peer own interactions. Canvas and Sensor are core classes representing the IoT device, which is the mobile phone. The Sensor responsibility is to collect the gyroscopical and positional (acceleration) data from the mobile phone, where the Canvas transforms the data to canvas drawing visualization. These classes are primarily used in UserDraw and Guess in order to incorporate the IoT interaction in the game flow. The Canvas data is stored in the DataBase. WordDictionary is a simplistic and specifically designed class which is responsible for a particular aspect of the game flow, which is the generation of random words. One of the core classes is the SignalingServer. SignalingServer is responsible to make the communication between the peers being possible to begin with. Further information about this core aspect will be provided in technical detail under the implementation chapter, but a more overview aspect in the general architecture chapter.

4.1.3 Sensor Overview

The sensor system in Airbrush allows users to draw by moving their phones. It collects motion data from the phone's accelerometer or gyroscope and translates it into brushstrokes on the digital canvas. This sensor data is broadcasted from the drawer to all other players using WebRTC, ensuring a consistent visualization for everyone.

We opted in Airbrush against using MQTT as a communication protocol due to its reliance on a central broker, which conflicts with the goal of decentralization. WebRTC provides a more suitable alternative, supporting direct P2P communication with built-in encryption and better scalability for real-time data transmission.

However, sensor inaccuracies posed significant challenges. Small errors in acceleration measurements accumulated over time, leading to noticeable drift in the drawings. Noise in the raw data also made it difficult to achieve smooth brushstrokes. To address these issues, the system employed noise filtering and smoothing techniques, though this came at the cost of reduced responsiveness. Regarding our product structure, we will provide with a UML diagram which shows the relation between the classes.

4.1.4 General Architecture

4.1.4.1 *User connecting to the other peers and establish communication*

The concepts discussed above provide an overview of the architecture based on our code implementation but do not address the broader system architecture or conceptual P2P (Peer-to-Peer) communication principles. Our program is hosted on a Digital Ocean cloud server, which facilitates user connections to a specific signaling server. The cloud server is primarily responsible for initializing peer connections and enabling communication through the `SignalingServer` class. After the initial setup, peers communicate directly with each other, downloading the necessary scripts locally and becoming independent of the cloud server. The cloud server is only needed again when a new user joins the network to exchange metadata for connection setup. More specifically, the `SignalingServer` on the cloud facilitates peer discovery and connection establishment. When multiple peers join, the signaling server enables them to exchange metadata and establish direct connections. Once these connections are established, communication between peers occurs directly, bypassing the cloud server entirely. The signaling process involves exchanging key metadata such as SDP (Session Description Protocol), which describes session parameters like media capabilities, codecs, and network information. While SDP defines how the connection should be established, it does not directly create the connection. Establishing a direct communication path requires additional steps, including the exchange of ICE (Interactive Connectivity Establishment) candidates, which are gathered using STUN or, as a fallback, TURN. The task of achieving direct peer connectivity is more complex than it might initially seem, primarily because the modern internet relies heavily on client-server architectures, which differ significantly from the decentralized nature of P2P communication. This deviation introduces challenges that require a deeper understanding of the underlying theoretical aspects of P2P networks. To enable direct communication between peers, we use WebRTC (Web Real-Time Communication). WebRTC facilitates direct communication by helping peers traverse NATs (Network Address Translations). NAT traversal is crucial because of the restrictions imposed by NATs and firewalls. Another option we had was the MQTT broker, but we simply chose the WebRTC, mainly because the MQTT broker is slower and based on a central broker in terms of message communication, whereas WebRTC is used to create a more P2P communication, being much less reliant on a specific single point of failure. On top of providing the possibility of optimal P2P communication, it has, by default, build in encryption for the data channels between the peers, so no man in the middle attack can occur while two peers are trying to communicate with each other given a communication channel. In order to have secure MQTT, the developer needs to provide a TLS setup between the broker and

the peer. The WebRTC accepts also from low to high data transfer to be send across the communication channels, something which the MQTT broker is quite limited. Therefore, for initializing the communication between P2P, we chose WebRTC instead of something like MQTT. An example of how NAT restrictions impede direct communication can be seen with two peers, A and B, both having private IPs. When these peers communicate through a NAT, their private IPs are mapped to public IPs by the NAT. However, neither peer knows the correct public IP and port to send data to, and NATs block unsolicited incoming packets. As a result, direct communication is impossible without additional mechanisms. Firewalls add another layer of restrictions, acting as traffic rules that determine which packets are allowed based on factors like source, destination, protocol, or port. Firewalls often block traffic that does not conform to predefined patterns. For example, university networks typically enforce stricter firewall policies than general-purpose home or business networks, potentially blocking P2P traffic. To traverse NATs, WebRTC uses STUN servers. A STUN server helps a peer discover its public-facing IP address and port as seen from outside the NAT. It also identifies the type of NAT the peer is behind. This information enables peers to create NAT mappings and establish direct connections. In cases where NAT restrictions are severe, such as in highly restrictive environments like university networks, WebRTC falls back to using a TURN server. The TURN server relays traffic between peers, ensuring communication even when direct connections are not possible. However, using a TURN server introduces additional latency and bandwidth costs. Technically, communication via TURN is no longer direct P2P, as messages are sent to the TURN server, which then forwards them to the intended peer. This reliance on a centralized server partially undermines the decentralized nature of P2P communication. Fortunately, most common networks, such as home or general-purpose business networks, have less restrictive NAT configurations. As a result, TURN servers are not frequently used, and WebRTC is typically able to establish direct connections using STUN.

4.1.4.2 Given communication established, general P2P architecture explained

Now that we have discussed about the technical details about the architecture behind the initialization of the P2P network, we can discuss about the actual P2P network itself. Because of a rather simplistic game and rather small P2P network, we omitted to use optimized algorithms in order to send data between peers. Even though we haven't used complex algorithms, our architecture is based on a specific P2P architecture, which is the Structured P2P architecture. All peers know each other and have channels made using the WebRTC, which makes our network being structured. Given no complex channel communication, we don't provide a DHT (Distributed Hash Table), we just

send data via the channels and save via peer DataBase the data the peer receives. The channel is secured via DTLS (Datagram Transport Layer Security), ensuring confidentiality, integrity and authenticity of the data being transmitted for the data being transported, effectively mitigating the Man in the Middle attacks. Another reason for picking structured P2P approach is due to its scalability and the predictable performance, given that we know where the packets are sent and to whom, smaller risk of having packets missing.

4.1.4.3 In depth explanation of the message flow from the architecture:

We discussed about the peers initialization and the general P2P architecture used, as well the arguments for using it, and now we want to go into details about the communication flow between the peers. For better explanation, we will assume that our network starts initially with 6 peers. Given that our game starts and we are at the first round, we use a specific algorithm (more about in the implementation section) to select a leader peer for each round which has the responsibility of broadcasting current data to the new coming peers, as well as constantly broadcasting data about the time remaining. The argument behind having a leader synchronizing the time is due to device having different clocks and therefore the risk of delays exists, leading to incorrect time values across the peers. The argument behind having a leader to deliver the current data for the new peers is due to its simplicity and ensure consistency, ensuring that the new peer will not receive duplicate data. The problem behind our approach is the single point of failure and the leader overhead, but we can argue for both of the scenarios and why they aren't a big concern. We start by arguing why single point of failure is not a concern for us. Single point of failure means that if a peer is login out or disconnects due to various reasons, the communication flow is drastically affected, requiring time and computational power in order to get back on track. In our case, this is not the case! The extra responsibilities that the leader have are concerning the canvas data distribution, timer data distribution and round data distribution. Logically speaking, the leader is the user who draws on the canvas, the drawer. Abstractly speaking, if the drawer leaves the game, it means that no one is capable of drawing, therefore that particular round is irrelevant. More concretely, if the round is irrelevant, we don't care about the data the leader was responsible and want to freshly start a new round where a new word is selected and the canvas is reinitialized. Every each round, a new leader is selected, after a while having all peers from the network being leaders, therefore no peer has more rights or more power than the others. Let's argue for leader overhead. The leader overhead is not a concern to us because of our initial assumption of the P2P network. As we stated previously, we want a simple and small network, meaning that the leader will need to broadcast data to maximum 15 peers, a small P2P

network. Again as previously, at the end, no peer will averagely have much more overhead than others, given that the leader role is shifting from peer to peer. The other peers that are not the leader are the one who are guessing the word, having the same HTML structure. The communication between the other peers is straightforward. If you interact with the interface, you broadcast your interaction with others. The other peers receive the broadcast and update their interface accordingly. In order to avoid race conditions, for each message send between the peers, especially if broadcasting, we use timestamps. More about it in the implementation section.

4.1.4.4 Canvas architecture. How do we transmit data from sensor to canvas

The most complex data distribution is the canvas. Given our previous explanation of communication flow, we want to go into more details about the most important data which is send through the channels, the canvas data. We start by explaining the canvas design and then explaining how the canvas receives the sensor data. We will not explain the canvas in terms of peers distribution because, conceptually, the distribution of the canvas in the communication channels is the same. The technical part behind the canvas data distribution and canvas general features will be explained under the implementation section

The canvas serves as a centralized hub, managing all data associated with it, including the processes of rendering and animation. These operations leverage the advanced capabilities provided by the three.js library to ensure seamless and dynamic updates to the ever-evolving visual environment. As the foundation of our drawing game, the canvas is designed to support air gesture drawing, which lies at the heart of the gameplay experience.

Players will have access to an array of tools tailored to enhance their creative expression. These tools include a brush for drawing, an eraser for removing specific parts of their work, and an undo function that reverses the most recent action taken. Additionally, a clear tool allows players to reset the canvas entirely. To facilitate personalized interactions, the system will utilize a JavaScript Map object to associate each line drawn with the specific player responsible for it. This mapping not only enables features like player-specific undo functionality but also lays the groundwork for future enhancements, such as a redo mechanism. Similar to the widely recognized Ctrl+Z functionality in most software applications, redo would allow players to reapply previously undone actions, effectively providing the opposite effect of the undo feature.

The canvas's functionality is further enriched by a dedicated support class called Line. This class is responsible for encapsulating three.js attributes while maintaining references to the points that define each line. By organizing data in this way, the Line class simplifies the

modification of line properties, streamlining interactions and paving the way for more sophisticated features in the future.

4.1.5 Implementation

4.1.5.1 *Non P2P related implementation*

We will start by explaining what the non core classes are used for in our project, to get a better understanding on the dynamic of the website, as well as understanding how the game change the role of the user, from being on the lobby, the userDrawer (leader) or the guesser. This chapter is meant to be a small explanation for each of the non core classes we use, since they didn't add that much in our learning regarding the P2P implementation, Cloud computing or regarding the IoT implementation. We will start by briefly explaining what the Spa class is and what it's relevance is for our project, then we will briefly talk about the Lobby, Database and very briefly about the WordGueser class. The Spa is actually a technique officially called Single Page Application, which the core concept is to enable the developer to have an application/website to be consistent of only one main HTML file. The main idea is that we have a start.html, which inside it we have div element called "appContainer". Given the user interactivity with our website, the appContainer changes accordingly. There are 3 contents the appContainer can shift towards, being the "lobby.html", the "guess.html" and "userDraw.html" content. When accessing the website, the user is has in his own "appContainer" the "lobby.html", then by writing his username and pressing on the play button, he actively initiates the communication and is redirected using the Spa class into the "guess.html" if there are peers in the network, or to "userDraw" otherwise. The idea is that the leader is the one who joined the earliest, making yourself the earliest to join if you are the only one in the server. When the game round ends, the Spa makes sure to update your state in the "appContainer" based on the leaderSelection algorithm, where if the leaderSelection algorithm selected you to be the drawer, then Spa will redirect you to have the "userDraw.html" data in the container, as well as the game having you to initialize the UserDraw class. The Game class has it's primary task of initializing one of the following classes "Guess", "UserDraw" and "Lobby". It used the Spa for loading the html part, as well as the CSS style. When the game mode is changed, we simply load an instance of that class representing the game mode, calling its init method which systematically calls the methods from that respective class. All the methods are based on event listeners, so multiple asynchronous called can happen via the "emit()" and "on()" methods (are explained in the communication implementation). An example will be to have the init method from the Guess class. In the init method, we have for example

`addChatBox` method which is an eventlistener that constantly listens for other peers adding something to their chat or having the peer locally adding messages on their chatBox. The Database class is actually only conceptually, since we store the data in the Communication class, but regardless, we will now explain what temporal data we store for each peer while they are playing the game. We are storing data regarding the score for each user, the map `userScoreMap`, a map called `listOfTheUserWhoGuessedTheWord`, the word dictionaries, the current chatlist data, peers data, `userPeerMap`, initial timer, data channels, leader word, round value, leader. The `listOfuserWhoGuessedTheWord`, `userScoreMap` and `userPeerMap` are the map data structures. The `listOfUserWhoGuessedTheWord` is a map where keys are the users and the value is the time. This is used to find when a specific user guessed the word. The `userScoreMap` is a map where key is the user and value is the score. Each entry is updated after each round, given that the user guessed the correct word. The `userPeerMap` is a map that simply keeps track of the users peer Id and their username. Used for updating the user list. Word dictionary is an object instance of the `WordDictionary` class, which it's used to get the dictionary with all the possible words and then, per round, shuffle for four specific words from which the `userDrawer` (leader) selects one of them. The chatlist contains the data of the messages from both the current peer, but also from the other peers, in the order the messages were sent (race condition possible, unfortunately) Peers data is a set where we store all the peer Id' of the peers that are connected to the network. Initial timer represents the timer the `userDrawer` starts its countdown from. Data channels are the secured directly connecting channels between the peers. We have a list with all the data channels the current peer can communicate to, which is $n-1$ number of channels given n peers in the network (you don't talk to yourself this time) Leader word is the word which is initially null, then the `userDrawer` selects a word, asynchronous updates it to a string value, then broadcast it to the guess peers. Round value is the current round the game is on. This is modified only by the `userDrawer`, the leader. The leader data is the ID of the leader. Each peer knows this information. This is useful in situations in which a peer takes different actions if a packet was sent by a leader then if it was by a guesser.

4.1.5.2 *Communication implementation:*

Given our more theoretical explanation of the system, providing explanation of its overall architecture, now let's go into more deep details and explain the code we used for its implementation. We will start by explaining the the initialization of the communication code, then chose to talk about the message distribution between the peers, shortly about the rest of the methods, then we also explain how the sensor and canvas work individually, then explaining how they work together in

the communication class, allowing us to distribute information about sensor and canvas to the other peers. Towards the end, we will briefly explain about the implementation regarding the other classes, due to their non relevance to the P2P architecture. Firstly, the signalingServer class starts with the following initializations:

Listing 4.1: signalingServer setup

```

1
2 const http = require('http');
3 const express = require('express');
4 const socketIO = require('socket.io');
5 const cors = require('cors');
6
7 const app = express();
8 app.use(cors());
9
10
11 const server = http.createServer(app);
12 const io = socketIO(server, {
13   cors: {
14     origin: "https://oscars-iot-gateway.mo00.com",
15     methods: ["GET", "POST"],
16     credentials: true
17   }
18 });
19
20 app.use(express.static('html'));

```

A very important aspect we can observe is already in the first line, where we use HTTP instead of HTTPS. Technically speaking, we have secured HTTPS connection via the Nginx, which is used as a Proxy to our singnalingserver which is on http. We let Nginx do the complex secure HTTPs connection due to it's already optimized design. In lines 3, 10 and 11-17, we initialize the server and socket, where the main idea is to initialize the singalingserver, as well as having a possibility to have each peer communicate with the server, a client server initial approach, using sockets. As explained before, the initial communication is between each individual peer and the signalingServer, in order to gather information about each peer from the signalingServer.

Listing 4.2: signalingServer class

```

1   const peers = {};
2
3
4 io.on('connection', (socket) => {
5   console.log('A peer connected:', socket.id);
6
7
8   // Register the peer with a unique ID
9   socket.on('register', (peerId) => {
10    socket.peerId = peerId;

```



```

11     peers[peerId] = socket;
12     console.log('Peer registered with ID: ${peerId}');
13
14
15     // Emit the updated peer list to all peers (including the newly
16     // connected one)
17     io.emit('Peer-Connected', Object.keys(peers));
18
19
20     // Relay signaling data between peers
21     socket.on('signal', (data) => {
22         const { to } = data;
23         const targetSocket = peers[to];
24         if (targetSocket) {
25             targetSocket.emit('signal', data);
26         } else {
27             console.log('Peer ${to} not found');
28         }
29     });
30
31
32     // Handle disconnection
33     socket.on('disconnect', () => {
34         if (socket.peerId) {
35             delete peers[socket.peerId];
36             console.log('Peer disconnected: ${socket.peerId}');
37             io.emit('Peer-Disconnected', Object.keys(peers)); // Notify all
38             // peers of updated peer list
39         }
40     });

```

Let's now go over more of how exactly each peers actually receive information regarding the other peers. Given the peer connection with the signalingserver using the socket, we have a global peers list for the signalingServer which is constantly updated when a peer disconnects or connects to the server. Each peer is identified with a random generator ID hexadecimal number and when the connection from each individual peer occurs, we can see in line 14 an emit of 'Peer-Connected' message broadcasted to all peers, containing the list of peers. More concretely, a peer joins and sends a connection emit, the server adds it to it's global list of peers, then broadcast the list of peers as type 'Peer-Connected' emit message. For 'Peer-Disconnected' emit, the same procedure is effectuated, but instead of adding a peer, we remove the peer which disconnected from the network. The signaling transmission, from line 18-26, is about helping the peers signal between each other. The signaling part is where we introduce the WebRTC concepts we talked in the previous chapter. The difference between the signaling type and both connection and disconnection is that we don't broadcast the signaling, but simply take the signaling request from one peer and send it to the target peer the initial peer wants to initialize the signaling. We will now explain the disconnec-

tion, connection and signaling from the peers perspective. We will start by providing a short explanation for 'Peer-Connected' and 'Peer-Disconnected'. Given that the signalingServer identifies a peer being disconnected or connected, we want to ensure for the other peers that they know that that particular peer disconnected or connected to the network. More concrete, the churn in the P2P network is unfortunately controlled by the signalingServer, so if the signalingServer is attacked or not working, the peers cannot identify a peer who wants to join the network and the channels with a disconnected peer will not be deleted, having peers sending messages to a disconnect peer. If a leader is disconnected, we ensure that the rest of the peers receive this information from the server, all peers running the leader selection algorithm and the newly leader will update the round, reset the data and tell the other peers that he is the new peer. Given peer connection, the important aspects are that each individual peer, when receiving info about new peers, save the peers ID and their usernames, and if the peer length is 1 (signalingServer sends the list of all the peers from the server, including yourself), then you have the possibility of running the leader algorithm and will make you the new leader, given that the network only have one peer. Given a new peer connected, each peer also makes sure to create a peer connection, using the method `createPeerConnection()`, to the newly connected peer. Now, let's go into detail about the signaling part from the peer perspective, as well as going into more details about the method `createPeerConnection()`. The script for the signaling part is:

Listing 4.3: signalingServer ICE server

```

1
2 this.socket.on('signal', async (data) => {
3   const { from, to, type } = data;
4   if (to !== this.peerId) return;
5
6   if (!this.peerConnections[from]) {
7     this.createPeerConnection(from, false, false);
8   }
9
10  const peerConnection = this.peerConnections[from];
11
12  if (type === 'offer') {
13    await peerConnection.setRemoteDescription(new RTCSessionDescription(
14      (data.offer)));
15    const answer = await peerConnection.createAnswer();
16    await peerConnection.setLocalDescription(answer);
17    this.socket.emit('signal', {
18      type: 'answer',
19      answer: peerConnection.localDescription,
20      from: this.peerId,
21      to: from,
22    });
23  } else if (type === 'answer') {

```

```

23     await peerConnection.setRemoteDescription(new RTCSessionDescription
        (data.answer));
24   } else if (type === 'candidate') {
25     if (data.candidate) {
26       await peerConnection.addIceCandidate(new RTCIceCandidate(data.
        candidate));
27     }
28   }
29   });

```

The core idea is that when an individual peer receives the 'signal' message from the signalServer, it means that another peer wants to establish direct connection with the current peer. We check if that's the case, if the other peer really wanted to send the message to the current peer, via line 2-3. The signaling happens AFTER establishing the initial channel connection. The idea is that the signaling will allow them to communicate via the channels created, bypassing NATs restrictions. The idea behind "type === offer", which can be seen on line 11 is that given that the peer is the initiator, in our logic the peer which has the highest ID value, will be the one making an offer to the connected peer. The offer basically means that the initiator sends its capabilities and parameters for media transmission and data channels, basically SDP set of instructions. The offer is made when creating peer Connection, at method createPeerConnection(): (below)

Listing 4.4: communication ICE server code

```

1
2 if (isInitiator) {
3     peerConnection.createOffer().then((offer) => {
4         return peerConnection.setLocalDescription(offer);
5     }).then(() => {
6         this.socket.emit('signal', {
7             type: 'offer',
8             offer: peerConnection.localDescription,
9             from: this.peerId,
10            to: targetPeerId,
11        });
12    }).catch((error) => {
13        console.error('Error creating offer:', error);
14    });
15 }

```

When the offer is received by the target peer, he eventually receives the information from the sender peer (initiator), then the target peer sends SDP set of instructions, same concept. In order to explain the "candidate" type signal, we look below the createPeerConnection() script for where both peers send their ICE candidate:

Listing 4.5: Communication ICE server

```

1
2 const peerConnection = new RTCPeerConnection({

```

```

3         iceServers: [{ urls: 'stun:stun.l.google.com:19302' }],
4     });
5
6
7     . . . . . (After a couple of lines)
8
9
10    peerConnection.onicecandidate = (event) => {
11        if (event.candidate) {
12            this.socket.emit('signal', {
13                type: 'candidate',
14                candidate: event.candidate,
15                from: this.peerId,
16                to: targetPeerId,
17            });
18        }
19    };

```

The idea is we first define for the peerConnections the iceServers, where we had actually provided a free STUN server from google. We started to not provide a TURN server because we wanted to see how well it will go without the TURN. We will provide more clear explanations at the experiment sections. This happens for both the peers connected on the same communication channel. After initializing the STUN, we provide our candidates, then on line 23-26, the receiver of the candidates can make his ICE using the candidates (data as port and IP address), then tries to make the connection via STUN server. If fails, as you saw that we don't have a TURN server, we just get errors, but more about it in the Experiment chapter. We actually summed also what createPeerConnection() contains, because there is a direct relation between the signaling section and the actual connection initialization. We make first the connection, then we start signaling. When the signaling between the servers is on place, both parties receiving the ICE agents and successfully making the connection. Let's now talk about the setting up the fully connected channels, afterwards talking about a particular message exchange, the chat JSON packets exchange between peers given different circumstances. Below we have the setup data channel initialization:

Listing 4.6: Communication setupDataChannel

```

1
2    createPeerConnection(targetPeerId, isInitiator, initiallyNewOne) {
3
4
5        . . . . . (code related to signaling)
6
7
8        if (isInitiator) {
9            const dataChannel = peerConnection.createDataChannel('
10                accelerometerDataChannel');
            this.setupDataChannel(targetPeerId, dataChannel,
                initiallyNewOne);

```

```

11     } else {
12         peerConnection.ondatachannel = (event) => {
13             const dataChannel = event.channel;
14             this.setupDataChannel(targetPeerId, dataChannel,
15                                   initiallyNewOne);
16         };
17     }
18
19     . . . . . (code related to signaling)
20
21
22 }

```

Shortly explained, if the peer is the initiator, then the initiator will actually create the data channel, giving it a specific name. Our structure is based on having a channels between pairs of two peers, so given n peers, n^2 channels. The channel optimization is not required, given that we have few peers playing the game. If the peer is not the initiator, then we simply connect to the already created datachannel by the initialization. Given successful direct connection between peers, we are reaching the method called `setupDataChannel`. This method responsibility is to manage the types of messages that are distributed between the peers, having different types of messages given different circumstances. In the majority of the cases, we have two types of situations: The first situation is when you have peers currently in the game and sending recent messages between each other, because they know the state of the game, and the second situation is when a new peer joins the network and needs to receive the full data, then actively being able to get recent data and act as the rest of the peers. We will explain these procedures by looking at two scenarios, given that the other implementation scenarios are quite similar. The first scenario is the chat updates between peers, the second being the canvas scenario, and we will start with the less complex one, the chat updates. We will start by talking about the scenario where a new peer joins the connection, providing only relevant code for this:

Listing 4.7: Communication `setupDataChannel`

```

1
2 setupDataChannel(targetPeerId, dataChannel, initiallyNewOne) {
3     this.dataChannels[targetPeerId] = dataChannel;
4
5
6     dataChannel.onopen = () => {
7         this.dataChannelOpener.log('Data channel is open with peer:',
8                                   targetPeerId);
9
10
11         if (initiallyNewOne) {
12             const requestFullData = {
13                 type: 'full_request',
14                 id: this.peerId
15             };
16         }
17     };
18 }

```

```

14         };
15
16         dataChannel.send(JSON.stringify(requestFullData));
17     }
18
19
20
21     . . . . . (additional code)
22
23 }

```

Given that a peer connects to the P2P network, no matter if we focus only on the chat messages distribution, the same happens for all cases, the newly connected peer sends a request to the other peers of receiving full data. What happens, the newly peer sends that request to all peers, informing them, given their peer id, who is actually sending this request. The request is made as a JSON packet. The other peers, which are the receivers, react on the type “full_request” message packet and, in response, create a full response containing

Listing 4.8: setupDataChannel

```

1
2 if (receivedData.type === 'full_request') {
3     this.canvasLog.log("processing full request")
4     if (this.peerId === this.leaderID) {
5         const allResponse = {
6             type: 'full_response',
7             leader: this.leaderID,
8             chatList: this.chatList,
9             timer: this.getCurrentTimer(),
10            listOfUserWhoGuessedTheWord : Array.from(this.
11                listOfTheUserWhoGuessedTheWord),
12            peerUserScoreMap: Array.from(this.
13                peerUserScoreMap),
14            leaderWord : this.leaderWord,
15            round: this.round,
16            id: this.peerId,
17        };
18
19        let newPeerID = receivedData.id
20        this.emit('Send_current_canvas_to_new_peer', {
21            newPeerId : newPeerID})
22
23        this.dataChannelOpener.log("From full request, I send
24            full_response which contains chatList:", this.
25            chatList);
26
27        dataChannel.send(JSON.stringify(allResponse));
28    }
29 }

```

From this chunk of code we care in particular about the structure and the data from lines 3-7 and the `peerId` from line 13. A critical part is the head of the if statement “if(this.peerId === this.leaderID)” because it tells whom responsibility is to broadcast the data, the leader. The benefits and liabilities were discussed in the architecture section, so we will not repeat ourselves again. The leader then responds to the newly connected peer, sending the peer its current data regarding different aspects of the game like the current chatList, the word selected by the leader, the leader ID for better communication and the current time given the specific round. The newly connected peer, given that he receives the current data from the leader, can only be a guess object (the leader is always the user drawer), therefore the HTML interface is `guess.html` and the class instance is of type `guess`. Given these information, we can explain how the newly connected peer, using the `Guess` class, can update his local interface based on the data received from the leader. We will only consider chat related updates from `Guess`. The user starts by calling the `initGuess`, calling the methods `addChatBox` and `getChatBox`. The `getChatBox` method is the one activating given new peer.

Listing 4.9: `getChatBox` from `Guess`

```

1  getChatBox() {
2
3
4  . . . . . (code related to list of user who guessed the
      word, above )
5
6
7      let chatBoxAccumulator = document.getElementById("
          chatBoxAccumulator");
8      let chatBoxElementList = Array.from(document.
          getElementsByClassName("chatBox"));
9      let chatData = this.communication.getChatData();
10
11
12     const existingMessages = Array.from(chatBoxElementList).map(el =>
        el.textContent);
13
14
15     chatData.forEach(({ ID, username, message }) => {
16         let messageText = ""
17         messageText = `${username}: ${message}`;
18
19
20
21
22         if (!existingMessages.includes(messageText)) {
23             const messageDiv = document.createElement("div");
24             messageDiv.className = "chatBox";
25             messageDiv.textContent = messageText;
26
27

```

```

28         if (listOfUsersWhoGuessedTheWord.get(ID)) {
29
30             messageDiv.style.background = "lightgreen";
31             messageDiv.style.color = "white"
32         }
33
34
35         chatBoxAccumulator.appendChild(messageDiv);
36     }
37 });
38
39
40
41 chatBoxAccumulator.scrollTop = chatBoxAccumulator.scrollHeight;
42 }

```

The main concept is that we start by connecting to the HTML chatlist class so we have the possibility to visually add the data, but the most important aspect is on line 7, where given the data received from the 'full-response', regarding the chatlist, we take that data, save it in a local variable in order to directly display it on the interface. An important aspect is that the data is stored in the Communication class. We didn't manage to split the communication into two classes, the Database and Communication therefore, more concretely, when the user receives "full-response", the data will be stored in his Communication class fieldvariables, resetting when disconnecting. The idea is then to go through the chatData received from the leader and then append it on the html. Each chatElement is an object of structure: ID, username, message, where the idea is that we append the chats in the order the leader has. Given that our code calls this method per each round, we want to ensure that, via existingMessages, that it's initially empty in order to append the chatData, because we append only data given that the chatList in the HTML is initially empty, given that you are a new peer which didn't update his HTML before. We talked about the situation of when the peer is a new peer, and now let's shortly explain also what happens if the peer only needs the last message. We will not provide additional code from the Communication part, because the request concept is similar, only provide the code from Guess class. Below is the relevant code from Guess concerning on sending the last chat to the user:

Listing 4.10: addChatBox from Guess

```

1
2
3 addChatBox() {
4     let chatBoxAccumulator = document.getElementById("
5         chatBoxAccumulator");
6     chatBoxAccumulator.innerHTML = '';
7     let userInputContainer = document.getElementById("userInput");
8     let userInput = userInputContainer ? userInputContainer.
9         querySelector("input[type='text']") : null;

```



```

8
9
10
11 . . . . . (code relating to the guessing word from the leader)
12
13
14
15
16
17
18
19
20     userInput.addEventListener("keydown", (event) => {
21         if (event.key === "Enter" && userInput.value.trim() !== '') {
22             const message = userInput.value.trim().toLowerCase();
23
24
25 . . . . . (code relating to user who guessed the word)
26
27
28
29
30         let id = this.communication.getMyId();
31
32
33
34
35 . . . . . (code relating to user who guessed the word)
36
37
38         this.communication.setChatData(id, username, message);
39
40
41         let dataChannels = this.communication.getDataChannels();
42         const messagePacket = {
43             type: 'message_packet',
44             id: id,
45             username,
46             peerGuessedTheWord: guessChecker,
47             timePeerSentTheMessage: currentTimer,
48             mess: message
49         };
50
51
52
53
54
55
56
57         for (let targetPeerId in dataChannels) {
58             const dataChannel = dataChannels[targetPeerId];
59             this.communication.sendingThroughDataChannel(
60                 dataChannel, messagePacket);
61         }
62
63

```

```

64         userInput.value = '';
65     }
66 });
67
68
69     this.communication.on('newChatMessage', (chatMessage) => {
70         const { ID, username, message } = chatMessage;
71
72         const messageDiv = document.createElement("div");
73         messageDiv.className = "chatBox";
74
75
76         . . . . . (code relating to user guessed word and scoring system)
77
78         if (ID === this.communication.getMyId()) {
79             messageDiv.textContent = `(You, ${scoreValueForThisPeer})
80                                     ${username}: ${message}`;
81         } else {
82             messageDiv.textContent = `(${scoreValueForThisPeer}) {
83                                     username}: ${message}`;
84         }
85
86         chatBoxAccumulator.insertBefore(messageDiv,
87                                         chatBoxAccumulator.firstChild);
88         chatBoxAccumulator.scrollTop = chatBoxAccumulator.
89                                         scrollTop;
90         if (chatBoxAccumulator.childElementCount > 3) {
91             chatBoxAccumulator.removeChild(chatBoxAccumulator.
92                                             lastChild);
93         }
94     });
95 }

```

We will split our explanation into 3 sections: 1-5, 13-48, 49-69. On lines 1-5, we select from the elements in `html` the element with id `chatBoxAccumulator`, which will contain each element from the list of chat elements, and the element with id `userInput`, which is the input element where the users can insert their messages, updating directly the `chatBoxAccumulator`. From lines 13-48, the core idea is that we have an event listener on `userInput` where if the user presses on “Enter” and has a message in the `userInput`, the `messageText` will be stored in the `message` constant, saved in the local `chatData`, then broadcast it further to the other peers with the type: “message_packet”. If not in the `setupDataChannel`, we use the `sendingThroughDataChannel` which is responsible to serialize the message and send it as JSON at the `dataChannel` selected. The `dataChannel`, as explained previously, is created in `createPeerConnection` method(). For lines 49-69, we should first explain the methods “on()” and “emit()”. The main point is that you firstly create the method “on()”, which is an asynchronous method that does a specific action given the “emit()” method was activated. In this way, we ensure a chronological order of the methods in an event based manner. If `emit()` for a particular type is activated, then

on() event listener is activated instantly. Given this information, we can explain what happens on lines 49-69. The core idea is when the user updates its chatDatabase, there is an emit on "newChatMessage". Given the emit, on the "on()" event, we want to update the newly received/inserted chatMessage to be updated visually on the screen. We check on lines 57-61 if the message from the database is from our local database, so the peerID is matching with the current peer ID, or if it's received from the communication channel, as a response for the "message_packet" packet type, so from another peer. The argument behind this is to add a simple "(YOU)" text display if the message was send from the current user, in order to better distinguish between the current peer and other peers. From line 63-68, we simply wanted, for a better phone design, to only display three last messages. The good part between this implementation is that it saves space for a better mobile visualization, but the bad part is that given someone guessed the correct word, if the user doesn't write anything else from that point on, it can be harder to spot the guesser visually. The rest of the implementations, excluding the canvas, are the userbox, the word visualiser, the timer the round and the point calculation. The concepts for the userbox are quite similar to what we've done in the chatBox, so we will explain very briefly how it works. The userbox is running by constantly updating the peerUserNameMap which is a map where the key is the peerID, and the value is the username. If a peer joins the network, he automatically receives "userName_info" from the other peers, giving him the possibility to know what are the username for each peerID. If the peer is not new, he will still get the username from the peers he is connected to. This happens in the communication. After each "userName_info" packet receival, the peer updates its HTML interface locally, in the Guess class. The update consist of making a list div element where each userBox is appended. Each userName_info is equal to a userBox, where we simply update the userBox by having its innerHTML content being the username for each peerID connected to the network, where we have a pointing symbol next to the userBox if the peerId is the current peer I. The timer, word visualiser, the rounds and the score system are updated only by the leader, where the changes are broadcasted to the other peers, which are the guesser. The difference between those three implementation, in terms of package distribution actions, is on the frequency of the broadcast distribution. The time changer it's concerned about broadcasting to the other peers the current time the leader has, so for each change in the timer (1 second), the leader broadcast the newly updated timer to all the peers. We don't want to have each peer counting down the timer by themselves, because of the differences in delay each device has, resulting in having different timer values for each peer. We want to synchronize the timer for all peers, therefore the guess peers use the leader peers clock. This means higher

overload for the peer, but should not be a concern given that our game is designed to be played by a small number of users. The communication mechanism and packet distribution is mostly the same as for the previous example. If a new peer connects, he gets the time from the packet type “full_response” and if the peer is not new and needs to newly updated time, this is handled by waiting for a packet called “timer_update” which the leader will send every one second for each guess peer. The local update of the timer and the distribution of the timer happens inside the UserDraw class, in the method called startTimer, in a interval loop which ends when the new leader is selected, more specifically, when the round is updated. The round updates only when the timer is set to 0. This is managed by the leader, where he sends to the other peers a packet message called “round_packet”, containing the new Leader, which the previous leader calculates using the leaderSelector algorithm, and also the new round number. The receiving peers update their round number in their local database, as well as the new leader. The other peers need to know who the leader is, mainly for dealing with the new timer, word and score calculation for the newly updated round, but it’s also relevant for the new peers in order to know from whom to accept the “full_response”. When receiving the “round_packet” there is an emit call “RoundUpdated” for each receiver, which is used to change on the guess interface the round counter accordingly. The word visualiser is similar to the timer, where you have the leader broadcasting hints regarding the word selected, which the guesser needs to guess. The main idea is that the leader, via userDraw class, broadcasts via the communication to the other peers the hidden letter of the word selected, then for each word reveal that is effectuated given the wordDictionary algorithm, the leader broadcast it further to the other peers. The letter reveal is a simple algorithm which is based on having all the words being revealed when the timer is equal to 0. The packet broadcasted is the “word_updater”, where the user just emits “WordUpdater”, having the on() eventlistener on their Guess class locally updating their word updater html interface. If the user manages to guess the word, he sends the packet “message_packet”, containing a boolean called “guessChecker” which is a boolean value that is true or false, depending on if the user actually managed to correctly guess the word. If the user guessed the word, the leader receiving the “message_packet” will broadcast a packet called “perGuessedTheWord” to the other peers, so the other peers know that they need to update their chatBox accordingly, whereas the leader will stop broadcasting the iterative letter reveal and simply send him a single packet containing the full word. The leader will also send the guesser a packet of the score received, based on the time it took the user to guess the word. The score system was implicitly explained in the word visualiser explanation, so another explanation

is not required, it's functionality is very straightforward and doesn't add more to the communication architecture.

4.1.5.3 *Canvas implementation*

The canvas relies on the Line class to manage its drawing objects. To understand the canvas implementation, it is essential to first explore the Line class. A Line object serves as a wrapper around the three.js Line, specifically the THREE.Line class. This wrapper simplifies interactions with attributes that players need to access, such as the line's color and its vertices—an array of points that define the Line geometry.

The Line constructor initializes all necessary components but does not include any vertex data initially. It creates a THREE.Line object, which requires a THREE.BufferGeometry and a THREE.LineBasicMaterial. The constructor accepts a single argument, 'color', which is directly passed to the THREE.LineBasicMaterial. The 'color' argument can either be a hexadecimal value or a string representing a color name based on X11 color names.

The initialization of THREE.BufferGeometry involves using a Float32Array. The Line class maintains a reference to this array, primarily managing it through the 'addPoint' function. Since Float32Array is non-dynamic and requires explicit memory allocation, a constant 'preAllocationSize' is set during initialization with a default value of 3 entries. Each point corresponds to three attributes (x, y, and z) to accommodate 3D functionality. To track available memory for vertices, the 'maxSize' attribute is set to the value of 'preAllocationSize', while the 'size' attribute monitors the current number of vertices.

After creating a Line object, a starting point can be added using the 'startLineFor' function, which requires the initial point as an argument. Additional points can be appended to the Line using the 'addPointFor' function, which accepts an object with x, y, and z attributes. This function appends new vertices after the last point. If the size of the vertices reaches 'maxSize', the Line class performs a memory reallocation using an exponential scaling approach. This ensures efficient handling of both short lines and long, continuous strokes. The 'finishLine' function finalizes the Line by appending the last point.

The Line class is used exclusively within the canvas implementation. The Canvas class handles broader functionality and integrates with the Line class to manage drawings. The Canvas constructor optionally accepts a serialized JSON string to copy another instance's state. This string can be generated using the 'toJson' method.

The canvas maintains an array of Lines for all players, stored in a JavaScript Map object called 'players', where peer IDs serve as keys. Although the current game version does not support simultaneous drawing by multiple players, the canvas has built-in support for such functionality. The canvas attaches a three.js renderer to an HTML

'<canvas>' element, which houses a THREE.Scene object. All three.js objects must be added to this scene to render them.

Canvas functionality is managed through various methods. Players can be added using the 'addPlayer' function by passing their peer ID as an argument. Drawing a line involves three steps: 1. Call 'startLineFor(peerID, position, color)' to create a new Line object with the specified color and initial position. This function adds two points with identical coordinates to the new Line and appends it to the player's array in the 'players' Map. 2. Use 'drawFor(peerID, position)' to append additional points to the player's current Line. 3. Call 'finishLineFor(peerID, position)' to finalize the player's Line by appending the final point.

The Canvas class also provides additional tools for drawers. The 'clear' function creates a new canvas instance, effectively erasing everything in the scene. The 'clearFor' function erases only the Lines associated with a specific player. Lastly, the 'undoFor' function, accessible through a dedicated button, removes a player's most recent Line, effectively undoing their last action.

4.1.6 Evaluation

We will focus more on specific prototype scenarios regarding simple tasks, like identifying the users connected to the server, having the chat system working, the word reveal working, and then focus more on the canvas scenarios. We start by checking the simple scenario, having two peers connected, being identified in the signalingServer, and then being able to see each other messages in the chatList while guessing the word. We will not draw anything on the canvas, we only want to test more the identification scenario. We start by checking first if the signalingServer can identify the 3 connecting peers to the network and get the following output: (Peers connected to the game)

```
My current map of ID > users is Map(3) { "peer-5kpqyn36s" => {}, "peer-hk3peb1xs" => {}, "peer-xh6for7vt" => {} }
  size: 3
  <entries>
    0: "peer-5kpqyn36s" => Object { userName: "fox", status: "active" }
    1: "peer-hk3peb1xs" => Object { userName: "Aban_phone", status: "active" }
    2: "peer-xh6for7vt" => Object { userName: "babazinna", status: "active" }
  <prototype>: Map.prototype { ... }
```

Figure 4.2: peers connected to the game

As we can observe, the signalingServer successfully identified the peers connected to the server. We now check for the userDrawer peer-5kpqyn36s, if its userNameMap is correctly updated, as well as having the communication channels with the correct peers: ()

We can observe that the peer has the map being correctly implemented, having all the peers being registered, as well as their usernames. Given that the leader peer has its map correctly updated, we want now to check if the communication channels are established with

```

o oscar@airbrush-site:/var/www/airbrush/server$ node signalingServer.js
Signaling server running on port 3000
A peer connected: EBUdF4S8pwxZ1h2xAAAAB
Peer registered with ID: peer-5kpqyn36s
A peer connected: p7M68u7K17Ubgfo5AAAD
Peer registered with ID: peer-hk3peb1xs
A peer connected: XC-fbhU2-9T_4ngrAAAF
Peer registered with ID: peer-xh6for7vt

```

Figure 4.3: SignalingServer Logs connecting peers

the other peers, the guesser peers: (logs of data channels with other peers)

```

Data channel is open with peer: peer-hk3peb1xs
Data channel is open with peer: peer-xh6for7vt

```

Figure 4.4: logs of data channels with other peers

Where we can observe that the data channels are established. We now want to check if the guesser can identify each other messages in the network. We will provide two pictures of the interfaces of both peer fox and peer babazinna: (Demonstration of chat function peer 1. + Demonstration of chat function peer 2.)

Above we have babazinna starting by saying “hi” and then Aban_phone user responds with “hell”. In the left hand side, you can see the babazinna point of view, where his texts appears in the chat with the prefix “(You)”, as intended, reflecting that the message belongs to the device user. The same can be seen in the right phone. We can therefore see that the chat works as intended, both parties getting each others messages. Now we check if the userDrawer also gets the chat updates for both parties. We can see that happening below: (Message data exchanged)

From this we can conclude that the userDrawer received the correct messages in their correspondent order, having the “hi” message appear first, then “hell”, and, as well, the messages are directly mapped to the correct users. The second experiment is based on the drawing canvas, see if the canvas is working. We will have only two peers, where one of the peers is the drawer, and the other one is the guesser. We want to see if the guesser can see what the userDrawer is drawing on the canvas. (Drawing function with motion data; drawing peer, Canvas view from guessing peer, UML Canvas draw from guess perspective.)

From the three pictures of rockets, we can see that in the first picture we have the userDrawer, and in the two others we have the guesser. We can see that the userDrawer successfully broadcasted the rocket to the other peers, as well as both guesser have the correct iteration of the word revealer.



Figure 4.5: Demonstration of chat function peer 1.



Figure 4.6: Demonstration of chat function peer 2.

```
Received data from peer-xh6for7vt: { type: "message_packet", id: "peer-xh6for7vt", username: "babazinna", peerGuessedTheWord: false, timePeerSentTheMessage: 60,
  id: "peer-xh6for7vt",
  mess: "hi",
  peerGuessedTheWord: false,
  timePeerSentTheMessage: 60,
  type: "message_packet",
  username: "babazinna" }
  <prototype>: Object { ... }

Received data from peer-hk3pebixs: { type: "message_packet", id: "peer-hk3pebixs", username: "Aban_phone", peerGuessedTheWord: false, timePeerSentTheMessage: 60,
  id: "peer-hk3pebixs",
  mess: "hell",
  peerGuessedTheWord: false,
  timePeerSentTheMessage: 60,
  type: "message_packet",
  username: "Aban_phone" }
  <prototype>: Object { ... }
```

Figure 4.7: message data exchanged



Figure 4.8: Drawing function with motion data; drawing peer.

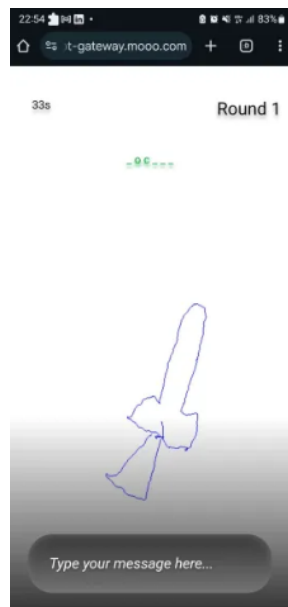


Figure 4.9: Canvas view from guessing peer.

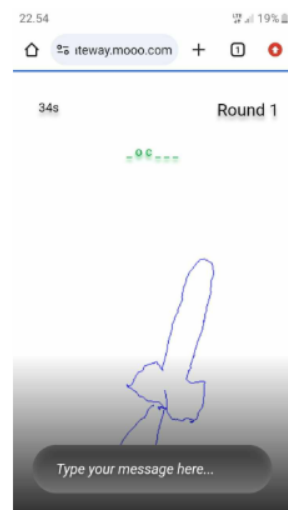


Figure 4.10: Canvas view from 2nd guessing peer.

5 | RESULTS

Airbrush accomplishes many of the things that it set out to do. The signaling server was able to identify peers connected to the server, and the peer map registers them all with their usernames. Communication was then established successfully in between the peers through WebRTS from this. They were able to communicate with each other, sending packages of line coordinates or chat messages in real time. The game loop itself also works fully and any peers joining in late is able to receive all the information of the round. And so, many of the aspects that we wished included in the project were made, but we also failed in some particular implementation designs and thus had to discard them. Compared with Scribbl.io, it achieves its base functionality with peers, a drawing game with scores, but without the polish of animation and character design and so on. This is also not something that we intended to include in the project. Moreover, the sensor we wanted to implement did not work as intended, so we compromised and chose one that was easier. That was the gyroscope. This doesn't, like the name suggests, exactly encapsulates drawing in the air.

6

DISCUSSION

6.o.1 Conclusion

Given the current implementation, we conclude that we managed to successfully implement some of the desired concepts, but failed as well in some particular implementation designs. We will start by explaining what went well, then shortly explaining what didn't went as we expected.

One of the main concepts that went well was a very reliable connection between peers, given that they weren't behind very restrictive NATs. As you saw in evaluation description, as well as in the video, the peers were able to connect with each other and successfully synchronize their interfaces. The message distribution worked generally well, having the chat system working as expected as well as being able to identify the users being connected to the network from a specific peer point of view, as well as successfully storing data about the updates that were made both by the current peer itself, but also by the other peers. One of the core concepts that went as expected are the canvas and sensor being able to work together and therefore let the user draw something meaningful on the canvas. As it can be seen in the Evaluation chapter, if the userDrawer is drawing something on the canvas, the othe peers are instantly provided with the correct canvas update. This shows that the userDrawer is able to identify the mobile phone movement, sensor data, translate the data into canvas line drawings, and then being able to successfully distribtue each update at a time, using the communication channels, to the guesser peers, as well as having the guesser peers being able to instantly updating their own local canvas data. Another core aspect regarding the canvas is that a newly joined peer is able to receive a `full_response` from a leader which is drawing actively on the game, being provided with the current canvas, as well as immediately being able to update its data if some canvas updates occur. Another benefit our project has is regarding to the WebRTC security mechanism. Given the communication channel, the message distribution across the peers is encrypted, therefore no other peer or adversary can interfere with a peers channel connections and specific message distribution across those channels. Given that we have talked about aspects that are working successfully in our prototype implementation, let's now talk about the elements that are not working and need ot be fixed in the future. One of the things that don't work as expected is the pointing distribution. What we have right now is that if the guesser correctly guesses the word,

the round ends instantly and no points are given to that particular peer. This is not something cumbersome to implement, but still a issue we have don't solved. This happens due to an testing attempt which took much more time than needed, then we didn't had time to bring the code to it's initial state, regarding to the point calculation section. Another issue with the code is more concerning the design section, where if you are connecting with a device with a bigger screen, it might be harder for you to see the messages send by the user. This issue is not concerning given the overall P2P structure, so therefore it's significance is quite limited Another issue is the time synchronization, a major issue. If we have multiple peers connecting to the server (20), due to overhead, the userDrawer has it difficult to successfully synchronize the peers and run smoothly. What happens is that the synchronization is correct, but the timer countdown is extremely slow, the message distribution is slow. We take this into consideration, since it's a major problem, and consider it for the future implementations. Another issue which can be problematic in the future is the race conditions that can occur between peers, more specifically on the chatBox list. If there are two peers who, at the same time, they write and send a message, there is probability that the peers receive the messages in a different order. Our initial consideration in order to solve the problem was to implement for each message, a timestamp, which should be constantly verified and ordered from the lowest to the highest timestamp. In this way, no peer will have different message orders due to race condition. Another issue which our code has is regarding the security part. Our website is vulnerable to Sybil attacks. If the same user tries to connect with multiple accounts, he can inject as many peers as his RAM memory allows him to, which is enough to kill the website and therefore make the game not playable.

BIBLIOGRAPHY

- [1] Stephanos Androutsellis-Theotokis and Diomidis Spinellis. “A survey of peer-to-peer content distribution technologies.” In: *ACM Comput. Surv.* 36.4 (Dec. 2004), 335–371. ISSN: 0360-0300. DOI: [10.1145/1041680.1041681](https://doi.org/10.1145/1041680.1041681). URL: <https://doi.org/10.1145/1041680.1041681>.
- [2] Ken Hinckley, Jeff Pierce, Mike Sinclair, and Eric Horvitz. “Sensing techniques for mobile interaction.” In: *Proceedings of the 13th Annual ACM Symposium on User Interface Software and Technology*. UIST ’00. San Diego, California, USA: Association for Computing Machinery, 2000, 91–100. ISBN: 1581132123. DOI: [10.1145/354401.354417](https://doi.org/10.1145/354401.354417). URL: <https://doi.org/10.1145/354401.354417>.
- [3] Urs Hunkeler, Hong Linh Truong, and Andy Stanford-Clark. “MQTT-S — A publish/subscribe protocol for Wireless Sensor Networks.” In: *2008 3rd International Conference on Communication Systems Software and Middleware and Workshops (COMSWARE ’08)*. 2008, pp. 791–798. DOI: [10.1109/COMSWA.2008.4554519](https://doi.org/10.1109/COMSWA.2008.4554519).
- [4] S. Loreto and S.P. Romano. *Real-Time Communication with WebRTC: Peer-to-Peer in the Browser*. O’Reilly Media, 2014. ISBN: 9781449371852. URL: <https://books.google.dk/books?id=RPNfAwAAQBAJ>.
- [5] *skribbl - Free Multiplayer Drawing & Guessing Game — skribbl.io*. <https://skribbl.io/>. [Accessed 08-12-2024].